# DBMS PRACTICALS

Four SQL Programs | MySQL

## Included programs

- 1. Create a database, apply constraints, and perform DDL/DML operations
- 2. Create related tables, add a foreign key, and implement referential integrity
- 3. Use different WHERE clauses and aggregate functions
- 4. Perform subqueries and simple JOIN operations

The examples use a consistent CollegeDB database with Department and Student tables where appropriate. Run each practical in order, or use the setup notes at the beginning of each section.

# Practical 1: Database, Constraints, DDL and DML

**Aim:** To create a database and table, apply PRIMARY KEY, UNIQUE, CHECK and NOT NULL constraints, and perform INSERT, UPDATE and DELETE operations.

## 1. Create the database and table

```
CREATE DATABASE CollegeDB;
USE CollegeDB;

CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Age INT CHECK (Age >= 18),
    City VARCHAR(50)
);
```

## 2. Insert records

```
INSERT INTO Student (StudentID, Name, Email, Age, City)
VALUES
(1, 'Rahul', 'rahul@gmail.com', 20, 'Ahmedabad'),
(2, 'Priya', 'priya@gmail.com', 22, 'Surat'),
(3, 'Amit', 'amit@gmail.com', 19, 'Vadodara');
```

## 3. Display all records

```
SELECT * FROM Student;
```

## 4. Update a record

```
UPDATE Student
SET City = 'Mumbai'
WHERE StudentID = 1;
```

## 5. Delete a record

```
DELETE FROM Student
WHERE StudentID = 3;
```

## 6. Add constraints with ALTER TABLE

Use these ALTER statements only when the corresponding constraint does not already exist. The table above already has these constraints.

```
-- Examples for a table that does not yet have these constraints:
ALTER TABLE Student
ADD CONSTRAINT pk_student PRIMARY KEY (StudentID);

ALTER TABLE Student
ADD CONSTRAINT uq_email UNIQUE (Email);

ALTER TABLE Student
ADD CONSTRAINT chk_age CHECK (Age >= 18);

-- MySQL syntax for NOT NULL:
ALTER TABLE Student
MODIFY Name VARCHAR(50) NOT NULL;
```

## Key points

- **PRIMARY KEY:** uniquely identifies each row and does not allow NULL.
- **UNIQUE:** prevents duplicate values in the constrained column.
- **CHECK:** restricts values according to a condition.
- **NOT NULL:** requires a value to be supplied.
- **DDL:** CREATE, ALTER, DROP. **DML:** INSERT, UPDATE, DELETE. SELECT is commonly classified as DQL.

**Result:** The database and table were created, constraints were applied, and data manipulation commands were performed.

# Practical 2: Foreign Key and Referential Integrity

**Aim:** To create parent and child tables, establish a foreign key relationship, and enforce referential integrity.

## 1. Create database and parent table

```
CREATE DATABASE CollegeDB;
USE CollegeDB;

CREATE TABLE Department (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50) NOT NULL UNIQUE
);
```

## 2. Create child table with a foreign key

```
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(50) NOT NULL,
    DeptID INT,
    FOREIGN KEY (DeptID)
        REFERENCES Department(DeptID)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
);
```

## 3. Insert records

```
INSERT INTO Department VALUES
(101, 'Computer Science'),
(102, 'Information Technology'),
(103, 'Commerce');

INSERT INTO Student VALUES
(1, 'Rahul', 101),
(2, 'Priya', 102),
(3, 'Amit', 101);
```

## 4. Display related records

```
SELECT Student.StudentID, Student.StudentName, Department.DeptName
FROM Student
INNER JOIN Department
ON Student.DeptID = Department.DeptID;
```

## 5. Test referential integrity

```
-- This fails because DeptID 999 does not exist:
INSERT INTO Student VALUES (4, 'Neha', 999);

-- This is rejected while students reference department 101:
DELETE FROM Department WHERE DeptID = 101;

-- Update cascades to matching Student rows:
UPDATE Department SET DeptID = 105 WHERE DeptID = 102;
```

**Explanation:** A foreign key requires each non-NULL child key to match a key in the parent table. ON UPDATE CASCADE propagates parent-key changes. ON DELETE RESTRICT prevents deleting a parent row that is still referenced.

## 6. Add a foreign key to an existing table

```
ALTER TABLE Student
ADD CONSTRAINT fk_student_department
FOREIGN KEY (DeptID)
REFERENCES Department(DeptID)
ON UPDATE CASCADE
ON DELETE RESTRICT;
```

Run the ALTER TABLE statement only if the foreign key has not already been created. Existing child values must match a parent key before the constraint can be added.

**Result:** A foreign key relationship was created and referential integrity was demonstrated.

# Practical 3: WHERE Clauses and Aggregate Functions

**Aim:** To retrieve records using WHERE conditions and calculate results using aggregate functions.

**Setup:** Use the Student table with StudentID, StudentName, DeptID and Marks columns. If the table already exists from Practical 2, add Marks once and populate it as shown below.

```
-- Run only if Marks does not already exist:
ALTER TABLE Student ADD Marks INT;

UPDATE Student SET Marks = 85 WHERE StudentID = 1;
UPDATE Student SET Marks = 90 WHERE StudentID = 2;
UPDATE Student SET Marks = 75 WHERE StudentID = 3;

-- Optional additional sample rows, if not already present:
INSERT INTO Department VALUES (103, 'Commerce');
INSERT INTO Student VALUES (4, 'Neha', 103, 88);
INSERT INTO Student VALUES (5, 'Karan', 102, 95);
```

If Practical 2 already inserted Department 103, or Student IDs 4 and 5, skip the matching INSERT statements to avoid duplicate-key errors.

## 1. Different WHERE clauses

### Equal to (=)

```
SELECT * FROM Student WHERE DeptID = 101;
```

### Greater than (>)

```
SELECT * FROM Student WHERE StudentID > 2;
```

### Less than (<)

```
SELECT * FROM Student WHERE StudentID < 4;
```

### AND

```
SELECT * FROM Student WHERE StudentID > 2 AND DeptID = 102;
```

### OR

```
SELECT * FROM Student WHERE DeptID = 101 OR DeptID = 103;
```

### BETWEEN

```
SELECT * FROM Student WHERE StudentID BETWEEN 2 AND 4;
```

### IN

```
SELECT * FROM Student WHERE DeptID IN (101, 102);
```

### NOT IN

```
SELECT * FROM Student WHERE DeptID NOT IN (101, 102);
```

### LIKE

```
SELECT * FROM Student WHERE StudentName LIKE 'R%';
```

### IS NULL

```
SELECT * FROM Student WHERE DeptID IS NULL;
```

### IS NOT NULL

```
SELECT * FROM Student WHERE DeptID IS NOT NULL;
```

## 2. Aggregate functions

```
SELECT COUNT(*) AS TotalStudents FROM Student;
SELECT SUM(Marks) AS TotalMarks FROM Student;
SELECT AVG(Marks) AS AverageMarks FROM Student;
```

```
SELECT MAX(Marks) AS HighestMarks FROM Student;
SELECT MIN(Marks) AS LowestMarks FROM Student;
```

For the five sample marks 85, 90, 75, 88 and 95, the total is 433, average is 86.6, maximum is 95, and minimum is 75.

## 3. Aggregate functions with WHERE

```
SELECT COUNT(*) AS StudentsInDept101
FROM Student
WHERE DeptID = 101;

SELECT AVG(Marks) AS AverageMarks
FROM Student
WHERE Marks >= 80;

SELECT MAX(Marks) AS HighestMarks
FROM Student
WHERE DeptID = 102;
```

## 4. GROUP BY and HAVING

```
SELECT DeptID, COUNT(*) AS TotalStudents
FROM Student
GROUP BY DeptID;

SELECT DeptID, AVG(Marks) AS AverageMarks
FROM Student
GROUP BY DeptID;

SELECT DeptID, AVG(Marks) AS AverageMarks
FROM Student
GROUP BY DeptID
HAVING AVG(Marks) > 85;
```

**Difference:** WHERE filters individual rows before grouping. HAVING filters grouped results, often using aggregate functions.

**Result:** WHERE conditions and aggregate functions were used to filter, count, total, and summarize student records.

# Practical 4: Subqueries and JOIN Operations

**Aim:** To use subqueries and combine related records using different SQL JOIN operations.

**Setup:** This practical uses Department(DeptID, DeptName) and Student(StudentID, StudentName, DeptID, Marks). If creating the tables from scratch, use the following schema.

```
CREATE DATABASE CollegeDB;
USE CollegeDB;

CREATE TABLE Department (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50) NOT NULL
);

CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(50) NOT NULL,
    DeptID INT,
    Marks INT,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
```

If the tables already exist from Practical 2 or 3, do not run CREATE TABLE again. Ensure the Student table has a Marks column for the examples below.

## 1. Sample records (only if the tables are empty)

```
INSERT INTO Department VALUES
(101, 'Computer Science'),
(102, 'Information Technology'),
(103, 'Commerce');

INSERT INTO Student VALUES
(1, 'Rahul', 101, 85),
(2, 'Priya', 102, 90),
(3, 'Amit', 101, 75),
(4, 'Neha', 103, 88),
(5, 'Karan', 102, 95);
```

Skip these INSERT statements if the same IDs already exist.

## 2. Subqueries

### Students scoring above the average

```
SELECT *
FROM Student
WHERE Marks > (SELECT AVG(Marks) FROM Student);
```

### Students in the Computer Science department

```
SELECT *
FROM Student
WHERE DeptID = (
    SELECT DeptID
    FROM Department
    WHERE DeptName = 'Computer Science'
);
```

### Student or students with the highest marks

```
SELECT *
FROM Student
WHERE Marks = (SELECT MAX(Marks) FROM Student);
```

### Students scoring more than Rahul

```
SELECT StudentName, Marks
FROM Student
WHERE Marks > (
    SELECT Marks
```

```
    FROM Student
    WHERE StudentName = 'Rahul'
);
```

### Students whose department ID appears in the Student table

```
SELECT *
FROM Student
WHERE DeptID IN (
    SELECT DeptID
    FROM Student
    WHERE DeptID IS NOT NULL
);
```

## 3. JOIN operations

### INNER JOIN — matching students and departments

```
SELECT Student.StudentID, Student.StudentName, Department.DeptName
FROM Student
INNER JOIN Department
ON Student.DeptID = Department.DeptID;
```

### LEFT JOIN — all students, with matching department if available

```
SELECT Student.StudentName, Department.DeptName
FROM Student
LEFT JOIN Department
ON Student.DeptID = Department.DeptID;
```

### RIGHT JOIN — all departments, including departments without students

```
SELECT Student.StudentName, Department.DeptName
FROM Student
RIGHT JOIN Department
ON Student.DeptID = Department.DeptID;
```

### CROSS JOIN — every student paired with every department

```
SELECT Student.StudentName, Department.DeptName
FROM Student
CROSS JOIN Department;
```

**Key points:** A subquery is nested inside another query. INNER JOIN returns matching rows; LEFT JOIN keeps every row from the left table; RIGHT JOIN keeps every row from the right table; CROSS JOIN returns all possible pairs.

**Result:** Subqueries and common JOIN operations were performed successfully using the Student and Department tables.